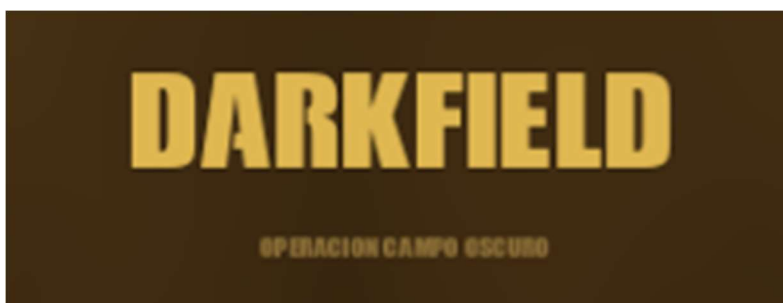


UNIVERSIDAD PRIVADA BOLIVIANA



Informe Final de Proyecto

Materia: Infografia

Docente: Jose Eduardo Laruta Espejo

Proyecto: Darkfield

Integrantes: Paul Timothy Kuno Serrano - Cod. 78533

Mayra Suzeth Rosas Saavedra - Cod. 77944

Fecha: 26 de junio de 2026

1. Introducción

Darkfield es un videojuego de sigilo y estrategia para un jugador, desarrollado en Godot 4 con GDScript, que fusiona la mecánica clásica del buscaminas con la tensión de la evasión en tiempo real. El jugador encarna a un soldado infiltrado tras líneas enemigas que debe limpiar un campo minado revelando casillas de forma deductiva, mientras evade a guardias enemigos o como nosotros le decimos cazadores dotados de un cono de visión dinámico e inteligencia artificial basada en máquina de estados. El proyecto incluye un modo historia de tres niveles con dificultad progresiva, mapas generados proceduralmente mediante autómatas celulares, un sistema de inventario e items con efectos especiales, un combate final contra un jefe final llamado General Karimi con fases, y un sistema completo de audio y opciones persistentes. El objetivo principal fue aplicar conceptos de la materia de infografía como renderizado 2D, iluminación, shaders, sprites sobre una arquitectura de software modular y orientada a objetos.

2. Motivación y Antecedentes

La motivación central de nuestro proyecto fue tomar una mecánica de juego reconocida y bien comprendida el buscaminas, además que es uno de nuestros juegos favoritos y reinventarla introduciendo presión temporal y cazador, transformando un juego de lógica pausada en una experiencia de sigilo y tensión constante. Nos interesaba demostrar que un género clásico puede revitalizarse combinándolo con sistemas modernos de inteligencia artificial y generación procedural. Además, fue también la fusión de dos ideas conjuntas de juegos que nos gustan.

Referentes que inspiraron el desarrollo:

- **Buscaminas (Microsoft, 1990):** base de la mecánica de revelado deductivo, números que indican minas adyacentes y banderas de marcado.
- **Metal Gear Solid:** referente del sigilo basado en conos de visión enemigos, estados de alerta y la necesidad de planificar rutas evitando la detección.
- **Call of Duty:** inspiración estética y narrativa además de ambientación militar, briefings de misión, tipografía y atmosfera bélica.
- **Roguelikes con generación procedural:** el uso de autómatas celulares para crear cavernas orgánicas proviene de técnicas populares en juegos del género como Spelunky o Dwarf Fortress.

La narrativa enmarca al jugador como un soldado que debe avanzar a través de territorio hostil, limpiar el camino de trampas y alcanzar el punto de extracción, enfrentando al final a un jefe militar enemigo con capacidad de invocar refuerzos.

3. Objetivos

Objetivo general

Desarrollar una aplicación interactiva en Godot 4 que aplique los conceptos de infografía 2D estudiados en la materia, sobre una arquitectura de software orientada a objetos, modular y mantenible, entregando un juego completo y jugable.

Objetivos específicos

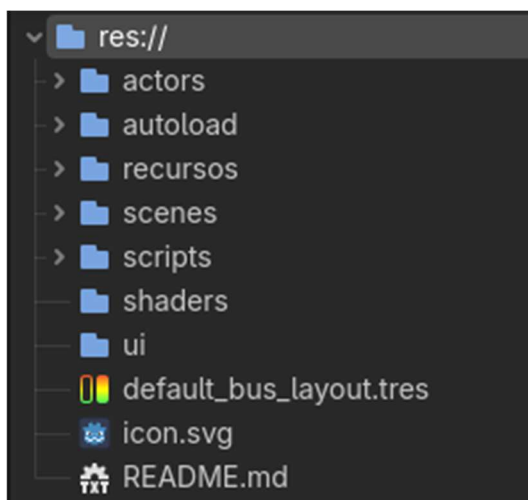
- Implementar la lógica del buscaminas como colocación de minas, cálculo de números, revelado en cascada y condición de victoria de forma desacoplada de la presentación grafica.
- Diseñar una inteligencia artificial para los enemigos basada en máquina de estados finita, con detección por cono de visión y pathfinding sobre la grilla.
- Generar mapas proceduralmente mediante autómatas celulares, garantizando conectividad total del espacio jugable.
- Construir un modo historia con progresión de niveles, sistema de inventario, items con efectos y un combate final contra un jefe con múltiples fases.
- Integrar elementos de infografía: renderizado por capas (TileMaps), iluminación 2D, shaders de fondo, niebla de guerra y dibujo vectorial dinámico del cono de visión.
- Incorporar audio, opciones configurables como el volumen, brillo, pantalla completa y persistencia de datos del jugador.

4. Descripción del Proyecto

4.1 Arquitectura general

El proyecto sigue una separación estricta entre la lógica del juego y su representación grafica. La lógica pura (mapa, tablero, generación procedural) se implementa en clases que heredan de `RefCounted` y no dependen de nodos visuales, lo que las hace testeables y reutilizables. La capa de presentación (escenas, actores, HUD) consume esa lógica. Dos singletons (autoloads) actúan como fuente de verdad global: `GameManager` centraliza el estado, la configuración de niveles, el inventario y la persistencia; `GestorAudio` centraliza música y efectos de sonido.

Jerarquía del proyecto:



Donde:

- **autoload/**: Contiene el `game_manager.gd`, `gestor_audio.gd`
- **actors/**: Contiene el jugador, cazador, boss, proyectil (`.gd` + `.tscn`)
- **scenes/**: Contiene menu, juego, historia, opciones, pausa, cámara
- **scripts/logica/**: Contiene `mapa.gd`, `tablero.gd`, `generador_dungeon.gd`, `item_mapa.gd`
- **recursos/**: Contiene sprites, tilesets, sonidos, fuentes

4.2 Patrones de diseño utilizados

- **Singleton (autoload):** GameManager y GestorAudio son instancias únicas accesibles globalmente, que mantienen el estado del juego y coordinan los sistemas.
- **Máquina de estados finita (FSM):** tanto los cazadores que tienen 5 estados como el jefe con 2 fases usan una FSM para gobernar su comportamiento.
- **Observador (signals):** la comunicación entre sistemas usa el sistema de señales de Godot como nivel_completado, jugador_murio, inventario_cambiado, desacoplando emisor y receptor.
- **Separación lógica/presentación:** las clases de lógica no conocen la capa gráfica, comunicándose a través de datos puros con coordenadas, arreglos.

4.3 Algoritmos desarrollados

a) Generación procedural de mapas (autómata celular)

Los mapas se generan partiendo de ruido aleatorio (42% de probabilidad de pared por celda) sobre el cual se aplican cuatro pasos de suavizado. En cada paso, una celda se convierte en pared si tiene más de cuatro vecinos pared y en suelo si tiene menos de cuatro. Este proceso transforma el ruido en cavernas orgánicas. Antes del suavizado se garantiza una zona segura alrededor del punto de inicio para que el jugador nunca aparezca encerrado.

b) Conectividad mediante flood fill (BFS)

Para evitar zonas abiertas inalcanzables, se ejecuta una búsqueda en anchura desde la celda de inicio que marca todas las celdas de suelo conectadas. Cualquier celda abierta no alcanzada se convierte en pared, garantizando que el 100% del espacio jugable este conectado.

c) Lógica del buscaminas y revelado en cascada

El cálculo de números cuenta, para cada celda de suelo, cuantas de sus ocho vecinas son minas, ignorando las paredes (que nunca contienen minas). El revelado en cascada se implementa con un flood fill iterativo en cuatro direcciones cardinales: al revelar una celda con numero cero se propagan sus vecinas. Se eligió la versión iterativa con cola en lugar de recursión para evitar desbordamiento de pila en mapas grandes, y se limitó la propagación a direcciones cardinales para que las paredes actúen como barreras reales del revelado.

d) Inteligencia artificial de los cazadores

Cada cazador es gobernado por una máquina de estados de cinco estados: NORMAL (patrulla con rutas aleatorias), SOSPECHANDO (ha detectado algo y se acerca cautelosamente), PERSIGUIENDO (caza activa), INVESTIGANDO (va a la última posición conocida) y REGRESA (vuelve a su punto de origen). Cada estado modifica velocidad, alcance y amplitud del cono de visión.

La detección combina tres condiciones: distancia dentro del alcance, ángulo del objetivo dentro del cono de visión, y línea de vista despejada verificada con un RayCast2D que se bloquea ante paredes. La persecución usa pathfinding por BFS sobre la grilla de celdas, eligiendo BFS porque garantiza el camino más corto en una grilla sin pesos. Un sistema de reservas de celdas evita que los cazadores se amontonen u ocupen la misma posición.

e) Jefe final con fases y proyectiles en arco

El General Karimi reutiliza el pathfinding BFS de los cazadores y agrega una máquina de estados de dos fases que se activa según su vida restante. Lanza granadas en intervalos e invoca soldados de refuerzo. Las granadas describen una trayectoria en arco calculada interpolando linealmente la posición entre origen y destino, sumando un desplazamiento vertical mediante una función seno para simular la parábola.

4.4 Elementos de infografía aprendidos en la materia

- **Renderizado por capas:** el mundo se compone de varios TileMapLayer (suelo, paredes, celdas reveladas) superpuestos.
- **Iluminación 2D:** cada cazador porta una luz (PointLight2D) que ilumina su entorno, junto a un nodo CanvasModulate global que crea la atmosfera nocturna.
- **Shaders:** el fondo de los menus usa un shader procedural animado que simula una tormenta de arena del desierto.
- **Dibujo vectorial dinamico:** el cono de visión de cada cazador se dibuja en tiempo real con la funcion `_draw()`, generando un poligono cuyo color cambia segun el estado de alerta.
- **Niebla de guerra:** las celdas no reveladas se cubren con un rectangulo semitransparente que se retira al ser descubiertas, limitando la informacion visible al jugador.

4.5 Sistemas de juego

- **Modo historia:** Contiene tres niveles de dificultad creciente con desbloqueo progresivo y guardado del avance mediante ConfigFile.
- **Modo partida rápida:** Partida clasica con niveles según tipo de dificultad elegida, hay 3 tipos fácil, medio y difícil, aquí no hay ítems.
- **Inventario e items:** radar de minas, boost de velocidad, congelado de enemigos, escudo, pistola y bazuka. Se obtienen en el mapa y al completar niveles, y persisten entre niveles.
- **Audio:** musica distinta por escena (menu, juego, jefe) y efectos de sonido para cada accion, gestionados por buses separados de Musica y SFX.
- **Opciones persistentes:** control de volumen de musica y efectos, brillo y pantalla completa, guardados entre sesiones.

5. Conclusiones y Recomendaciones

5.1 Conclusiones

El proyecto cumplió con los objetivos planteados se logro una aplicacion interactiva, jugable de principio a fin, que aplica multiples conceptos de infografia 2D sobre una arquitectura orientada a objetos y modular. La separacion entre logica y presentacion demostro ser una decision acertada, ya que nos permitio iterar sobre los algoritmos sin afectar la capa grafica. La fusion del buscaminas con mecanicas de sigilo resulto en una experiencia diferenciada y coherente que nos termino gustando mucho la verdad.

5.2 Retos y dificultades encontrados

- **Desbordamiento de pila:** el revelado en cascada y el pathfinding inicialmente eran recursivos, lo que provocaba stack overflow en mapas grandes. Se resolvio reescribiendolos de forma iterativa con colas (BFS).

- **Generacion procedural defectuosa:** las primeras versiones del automata celular producian espacios diminutos o desconectados. Se corrigio reordenando la garantia de zona segura antes del suavizado y ajustando la densidad de paredes.
- **Coordenadas invertidas:** la diferencia entre el sistema (fila, columna) de la logica y el sistema (x, y) del TileMap de Godot causaba que los actores aparecieran en posiciones espejadas; se resolvió fijando una convencion consistente de conversion.
- **Enrutamiento de entrada en CanvasLayer:** los nodos CanvasLayer no participan del pipeline de input de Godot, por lo que la pausa se centralizo usando consulta directa de acciones en `_process`.
- **Recursion en la muerte del jefe:** la transicion de victoria reactivaba la fase del jefe; se corrigio con una bandera de estado que evita el bucle infinito.

5.3 Recomendaciones y trabajo futuro

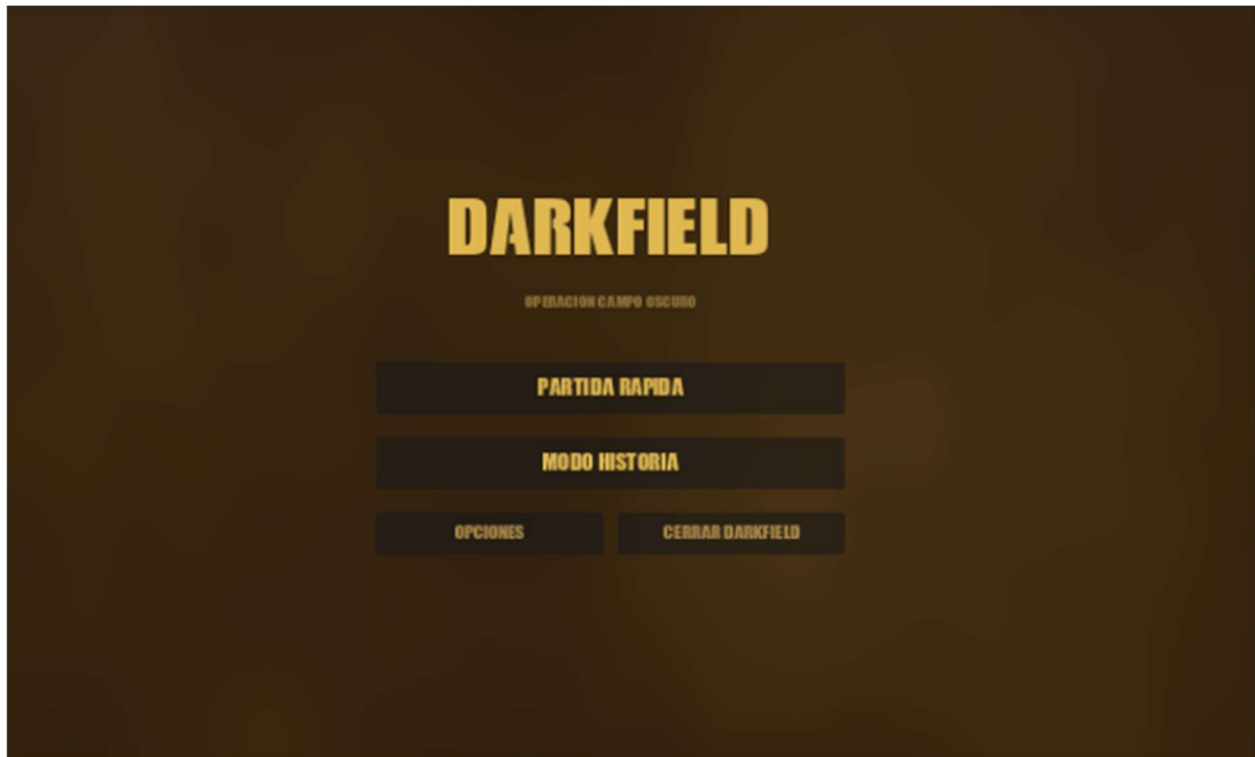
- Pensabamos en añadir un briefing narrado con voz al inicio de cada nivel del modo historia, al estilo de los informes de mision de los juegos belicos.
- Incorporar animaciones de transicion (aparicion y muerte de personajes) mediante shaders de disolucion.
- Implementar un sistema de dificultad adaptativa que ajuste el numero de enemigos segun el desempeño del jugador.
- Añadir mas variedad de items y un sistema de puntuacion con tabla de records.
- Optimizar la generacion procedural para mapas aun mas grandes y explorar otros algoritmos como salas conectadas por pasillos.

Repositorio del proyecto: <https://github.com/MayraSRS04/Darkfield>

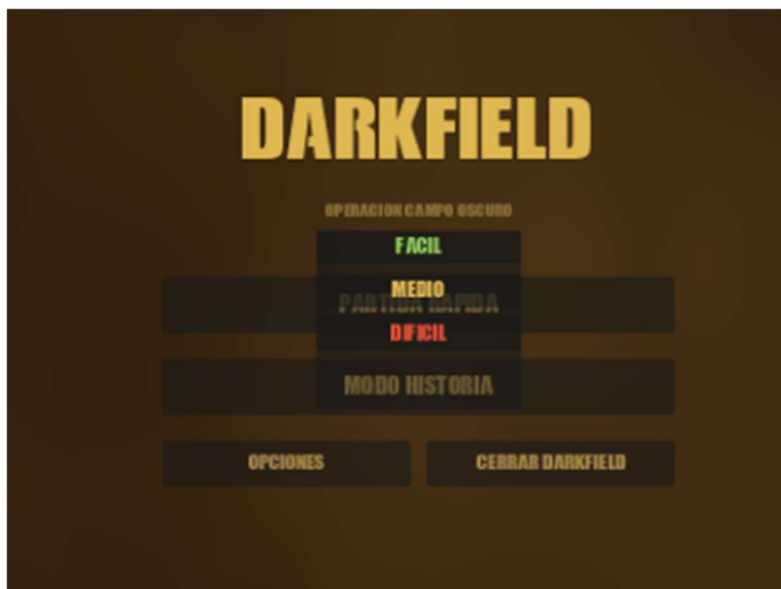
Motor: Godot 4.6 **Lenguaje:** GDScript

6. Anexos(Fotos)

Menu principal



Opciones de partida rápida



Un nivel de partida rápida



Pestaña modo historia

MODO HISTORIA

UN SOLDADO. UN CAMPO MINADO. SIN SALIDA.

NIVEL 1 LA EMBOSCADA

NIVEL 2 ZONA DE GUERRA

 NIVEL 3 EL ULTIMO BASTION

NUEVA PARTIDA

VOLVER

Un nivel de modo historia



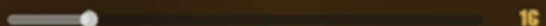
Pestaña de opciones

OPCIONES

BRILLO



MUSICA



EFFECTOS DE SONIDO



PANTALLA COMPLETA ☐

VOLVER